

Noumena Phase 1 Implementation Plan

CLI-first safe human-agent document loop

Prepared for Noumena

2026-05-29

Contents

- Executive summary
- Non-negotiable Phase 1 decisions
- Phase map
- Phase 1 success scenario
- Repository structure and vault format
- CLI contract
- Core data model and Noumena HTML v1
- Validation, revision, and block hash rules
- Document creation and patch protocol
- Conflict detection and review approval
- Write lock, atomic writes, SQLite, and rollback
- Canonical serialization and Phase 1 mutation HTML shapes
- Security and safety in Phase 1
- Testing, fixtures, acceptance criteria, and implementation sequence
- Error handling, open questions, future compatibility, and recommendation

1 Executive summary

Noumena is a local-first knowledge workspace built around structured, portable HTML documents and safe agent-authored semantic patches. The previous version of the implementation plan was architecturally sound, but too broad for the actual builder context: one builder, one AI coding partner, and Claude Code as the first agent calling shell commands.

This version rewrites the plan around a single Phase 1 goal:

Prove that Claude Code can safely create, inspect, patch, review, and roll back a Noumena-native HTML research note entirely through a standalone CLI, with no Electron app running.

Phase 1 is not an Electron app. It is not a Tiptap editor. It is not a full note-taking product. Phase 1 is the smallest correct implementation of the core promise:

1. A valid research-note@1 Noumena HTML file exists on disk.

2. Claude Code discovers the live document revision and block hashes through noumena doc info.
3. Claude Code submits a semantic JSON patch through noumena patch submit.
4. Noumena applies safe changes, queues unsafe changes for review, rejects invalid/conflicting changes, and never permits raw agent file rewrites.
5. Noumena snapshots before mutation and can roll back the latest applied patch exactly.
6. The entire loop is covered by headless integration tests and a demo script.

The canonical Phase 1 done condition is:

```
pnpm test:phase1
```

from a clean checkout, with no Electron process and no browser window. A second command:

```
pnpm demo:phase1
```

runs the end-to-end dogfood scenario.

2 Non-negotiable Phase 1 decisions

Area	Decision
Phase 1 product	Standalone CLI and core library only
First user	The builder
First agent	Claude Code calling shell commands
Phase 1 transport	Stdin/stdout CLI, not IPC, HTTP, MCP, or Electron
App dependency	No Electron required to create, patch, review, or roll back notes
Document format	Noumena HTML v1, constrained native subset of HTML
First document type	research-note@1 only
Agent writes	JSON semantic patch transactions only
Raw file rewrites by agents	Never supported
CLI command for patch submission	noumena patch submit, not apply
CLI stdout with --json	Exactly one JSON object plus newline
Conflict detection	Full document revision plus per-block hashes
Phase 1 roles	metadata, summary, notes, evidence, agent_events all required
Patch atomicity	Whole patch applies, queues, rejects, or no-ops; no partial apply
Rollback	Latest applied patch per document only
Rollback timestamp behavior	Restore snapshot exactly; do not stamp

Area	Decision
	updatedAt
Overwrite	No --overwrite in Phase 1
Editor	Deferred to Phase 3
Electron viewer/review shell	Deferred to Phase 2

3 Phase map

3.1 Phase 1 - CLI proof loop

Phase 1 ships:

- packages/noumena-core/
- packages/noumena-cli/
- Noumena HTML v1 format contract
- research-note@1 template
- patch JSON schema
- CLI result JSON schema
- fixture corpus
- SQLite derived index/history store
- vault write lock protocol
- snapshots and latest-patch rollback
- headless integration tests
- headless demo script

Phase 1 does not ship:

- Electron
- React
- IPC or preload scripts
- Tiptap or ProseMirror
- CodeMirror
- visual file tree
- omnibar or full-text search UI
- backlinks UI
- plugin architecture
- trusted interactive notes
- Markdown migration
- MCP integration
- Tauri/Rust runtime

3.2 Phase 2 - Minimal Electron viewer/review shell

Phase 2 wraps the working Phase 1 core in a read-only desktop shell:

- open vault
- show file tree
- render Noumena-native HTML in read-only preview
- watch file and .noumena/ changes
- show pending reviews
- show block diffs
- approve/reject queued reviews
- show history
- trigger latest-patch rollback
- use Electron secure defaults

No Tiptap editor in Phase 2.

3.3 Phase 3 - Comfortable editing

Phase 3 adds humane editing:

- Tiptap schema for Noumena-native blocks
- editable title, summary, notes, evidence, tags/status
- source inspector
- metadata panel
- dirty-editor conflict handling
- saves through the same noumena-core mutation pipeline

3.4 Later phases

Later phases may add search/backlinks UI, Markdown import/export, dashboard notes, static export, plugin architecture, trusted interactive notes, MCP, Python SDK, and Tauri evaluation. None of these are Phase 1 gates.

4 Phase 1 success scenario

The dogfood scenario is the product proof:

1. Builder initializes a vault.
2. Builder creates a research-note@1 document through the CLI.
3. Builder asks Claude Code to add evidence and update note status.
4. Claude Code calls noumena doc info to discover revision, roles, block IDs, and block hashes.
5. Claude Code writes a noumena.patch.v1 JSON file.
6. Claude Code calls noumena patch submit.

7. Noumena applies safe append/status operations or queues the patch for review.
8. Builder inspects queued review through noumena patch show --format=block-diff.
9. Builder approves or rejects through the CLI.
10. Noumena snapshots before every applied mutation.
11. Builder rolls back the latest applied patch.
12. pnpm test:phase1 proves all positive and negative cases headlessly.

The CLI is the Phase 1 product surface. The terminal diff is the Phase 1 review UI.

5 Repository structure

Phase 1 repository structure:

```
noumena/  
  package.json  
  pnpm-workspace.yaml  
  tsconfig.base.json  
  
packages/  
  noumena-core/  
    src/  
      ids.ts  
      pathSafety.ts  
      vault.ts  
      lock.ts  
      db.ts  
      parseNoumenaHtml.ts  
      canonicalSerialize.ts  
      canonicalSerializeBlock.ts  
      computeRevision.ts  
      validateDocument.ts  
      docCreate.ts  
      docInfo.ts  
      patchSchema.ts  
      evaluatePatch.ts  
      applyPatch.ts  
      writeTransaction.ts  
      indexDocument.ts  
      history.ts  
      reviewStore.ts  
      blockDiff.ts  
      doctor.ts  
    test/
```

noumena-cli/

src/

main.ts

output.ts

commands/

vaultInit.ts

vaultDoctor.ts

docCreate.ts

docInfo.ts

docValidate.ts

patchSubmit.ts

patchDryRun.ts

patchApprove.ts

patchReject.ts

patchRollback.ts

patchStatus.ts

patchShow.ts

test/

schemas/

noumena-html-v1.md

noumena-patch-v1.schema.json

noumena-cli-result-v1.schema.json

templates/

research-note@1.html

fixtures/

native/

malformed/

patches/

expected/

validation/

valid/

invalid/

scripts/

demo-phase1.ts

docs/

phase1-implementation-plan.md

Hard rule:

noumena-cli must contain command plumbing only. All parsing, validation, patching, indexing, history, locking, and serialization logic lives in noumena-core.

6 Vault format

A Phase 1 vault is a normal directory with portable HTML files and a .noumena/ sidecar directory for derived state and operational records.

```
MyVault/  
  research/  
    local-llms.html  
  
  .noumena/  
    index.sqlite  
    .gitignore  
    history/  
      patches/  
      snapshots/  
      rollbacks/  
    reviews/  
    logs/  
    locks/  
    stale/
```

Source-of-truth rules:

- The HTML file on disk is the canonical document source.
- .noumena/index.sqlite is derived state.
- doc info computes the live revision from disk.
- noumena:revision must not appear in HTML.
- Patch records, review records, history events, snapshots, and locks live under .noumena/.

Recommended .noumena/.gitignore:

```
locks/  
*.tmp
```

7 CLI contract

7.1 Global JSON output rule

Every command that accepts --json writes exactly one JSON object plus a trailing newline to stdout.

Exit code meanings:

Exit code	Meaning
0	Valid domain outcome; inspect JSON status
2	CLI usage error
3	patch JSON parse/schema error
4	vault/path/document error
5	internal failure

Domain rejections exit 0. Examples include `vault_write_lock_busy`, `document_validation_failed`, `missing_required_block_hash`, `target_block_locked`, `stale_review`, and `rollback_not_latest`.

7.2 Required commands

```
noumena vault init <vault> --json
```

```
noumena vault doctor <vault> --json
```

```
noumena doc create <vault> <doc-path> \  
  --type=research-note \  
  --title="..." \  
  --json
```

```
noumena doc info <vault> <doc-path> --json
```

```
noumena doc validate <vault> <doc-path> --json
```

```
noumena patch submit <vault> <doc-path> <patch.json> --json
```

```
noumena patch dry-run <vault> <doc-path> <patch.json> --json
```

```
noumena patch approve <vault> <review-id> --json
```

```
noumena patch reject <vault> <review-id> --json
```

```
noumena patch rollback <vault> <patch-id> --json
```

```
noumena patch status <vault> <patch-or-review-id> --json
```

```
noumena patch show <vault> <patch-or-review-id> --format=block-diff
```

7.3 Path rules

Document paths are always vault-relative POSIX paths:

- must not be absolute
- must not contain ..
- must end in .html for doc create
- must not resolve outside the vault after canonicalization
- must not be inside .noumena/
- symlink escape detection is required

Path escape attempts return exit 4 with `path_escapes_vault`.

8 Core data model

Phase 1 can use a small internal document model. It does not need to be the future Tiptap model, but it must be compatible with a future Tiptap schema.

```
type NoumenaDocument = {  
  schemaVersion: "noumena.html.v1";  
  documentType: "research-note";  
  documentId: DocumentId;  
  title: string;  
  metadata: NoumenaMetadataV1;  
  article: NoumenaArticle;  
  roles: Map<NoumenaRole, NoumenaBlock>;  
};
```

```
type NoumenaRole =  
  | "metadata"  
  | "summary"  
  | "notes"  
  | "evidence"  
  | "agent_events";
```

```
type ConflictPolicy =  
  | "append_only"  
  | "replace_requires_clean_base"  
  | "merge_text"  
  | "manual_review"  
  | "locked";
```

```
type NoumenaBlock = {  
  blockId: BlockId;  
  role: string;  
  policy: ConflictPolicy;  
  element: HTMLElement;  
  blockHash: string;  
};
```

```
type NoumenaMetadataV1 = {  
  title: string;  
  status: "draft" | "active" | "reviewed" | "archived";  
  createdAt: string;
```

```
updatedAt?: string;
tags?: string[];
aliases?: string[];
};
```

ID formats:

```
doc_[0-9A-HJKMNP-TV-Z]{26}
blk_[0-9A-HJKMNP-TV-Z]{26}
ev_[0-9A-HJKMNP-TV-Z]{26}
evt_[0-9A-HJKMNP-TV-Z]{26}
note_[0-9A-HJKMNP-TV-Z]{26}
```

Production should use a monotonic ULID-style generator. Tests must use deterministic IDs.

9 Noumena HTML v1

Noumena-native HTML is not arbitrary web-page HTML. It is a constrained, deterministic subset designed for human readability and agent-safe structural mutation.

Minimum research-note@1 shell:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Local LLM Memory Systems</title>
  <meta name="noumena:schema" content="noumena.html.v1">
  <meta name="noumena:document-id"
content="doc_01JZ9A7V6QK8M3X2T5N4R1B0CY">
  <meta name="noumena:document-type" content="research-note">
</head>
<body>
  <article
    data-noumena-document
    data-noumena-document-id="doc_01JZ9A7V6QK8M3X2T5N4R1B0CY"
    data-noumena-document-type="research-note"
    data-noumena-schema-version="noumena.html.v1">
    ...
  </article>
</body>
</html>
```

research-note@1 requires exactly one each of these roles:

Role	Required element	Required child	Allowed policies	Default
metadata	script[type="application/vnd.noumena.metadata+json"]	valid JSON	merge_text, locked	merge_text
summary	section	direct h2	replace_requires_clean_base, manual_review, locked	replace_requires_clean_base
notes	section	direct h2	append_only, locked	append_only
evidence	section	direct h2 plus direct ol	append_only, locked	append_only
agent_events	section	direct h2 plus direct ol	append_only, locked	append_only

No optional roles in Phase 1.

10 Research-note template

templates/research-note@1.html should be generated through the same serializer used for created documents. The template can include placeholders, but the created document must be real serialized output, not a hand-edited string.

Created note skeleton:

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Local LLM Memory Systems</title>
  <meta name="noumena:schema" content="noumena.html.v1">
  <meta name="noumena:document-id"
content="doc_01JZ9A7V6QK8M3X2T5N4R1B0CY">
  <meta name="noumena:document-type" content="research-note">
</head>
<body>
  <article
    data-noumena-document
    data-noumena-document-id="doc_01JZ9A7V6QK8M3X2T5N4R1B0CY"

```

```
data-noumena-document-type="research-note"
data-noumena-schema-version="noumena.html.v1">
<script
  data-noumena-block-id="blk_01JZ9A7Y8METADATA000000"
  data-noumena-role="metadata"
  data-conflict-policy="merge_text"
  type="application/vnd.noumena.metadata+json">
{
  "createdAt": "2026-05-29T23:00:00Z",
  "status": "draft",
  "tags": [],
  "title": "Local LLM Memory Systems",
  "updatedAt": "2026-05-29T23:00:00Z"
}
</script>

<section
  id="summary"
  data-noumena-block-id="blk_01JZ9A7Y8SUMMARY000000"
  data-noumena-role="summary"
  data-conflict-policy="replace_requires_clean_base">
  <h2>Summary</h2>
  <p></p>
</section>

<section
  id="notes"
  data-noumena-block-id="blk_01JZ9A7Y8NOTES00000000"
  data-noumena-role="notes"
  data-conflict-policy="append_only">
  <h2>Notes</h2>
  <p></p>
</section>

<section
  id="evidence"
  data-noumena-block-id="blk_01JZ9A7Y8EVIDENCE00000"
  data-noumena-role="evidence"
  data-conflict-policy="append_only">
  <h2>Evidence</h2>
  <ol></ol>
</section>
```

```

<section
  id="agent-events"
  data-noumena-block-id="blk_01JZ9A7Y8EVENTS00000"
  data-noumena-role="agent_events"
  data-conflict-policy="append_only">
  <h2>Agent Events</h2>
  <ol></ol>
</section>
</article>
</body>
</html>

```

The IDs above are illustrative; real generated IDs must satisfy the 26-character ULID-alphabet rule after the prefix.

11 Validation

validateDocument() is a core Phase 1 gate. It is called by doc create, doc info, doc validate, and patch submit.

```

type ValidationResult = {
  valid: boolean;
  patchable: boolean;
  fileClass: "noumena_native" | "raw_html" | "invalid_noumena";
  schemaVersion?: "noumena.html.v1";
  documentType?: "research-note";
  documentId?: string;
  title?: string;
  roles: RoleInfo[];
  errors: ValidationIssue[];
  warnings: ValidationIssue[];
};

```

```

type ValidationIssue = {
  severity: "error" | "warning";
  code: string;
  message: string;
  path?: string;
  role?: string;
  blockId?: string;
  policy?: string;
};

```

A document is Phase 1 patchable only when:

```
fileClass = noumena_native
schemaVersion = noumena.html.v1
documentType = research-note
validation.errors.length = 0
```

Warnings do not block patching.

11.1 Structural validation rules

Layer 1 applies to all Noumena HTML v1 documents.

Required:

- `<!doctype html>`
- `<html lang="en">`
- `<meta charset="utf-8">`
- non-empty `<title>`
- `<meta name="noumena:schema" content="noumena.html.v1">`
- `<meta name="noumena:document-id" content="doc_...">`
- `<meta name="noumena:document-type" content="research-note">`
- exactly one `<article data-noumena-document>`
- unique block IDs
- no duplicate roles
- known conflict policies
- metadata block with parseable JSON
- required metadata fields: title, status, createdAt

Errors include:

```
missing_document_article
unknown_schema_version
duplicate_block_id
duplicate_role
missing_metadata_block
metadata_invalid_json
unknown_conflict_policy
policy_not_allowed_for_role
forbidden_element
forbidden_event_handler
forbidden_srcdoc
forbidden_url_scheme
invalid_document_id
invalid_block_id
block_missing_id
```

block_missing_role
block_missing_conflict_policy
wrong_element_for_role
missing_required_role
evidence_missing_ol
agent_events_missing_ol
forbidden_revision_meta
unsupported_document_type

Warnings include:

title_metadata_mismatch
unknown_role
duplicate_html_id
evidence_item_missing_id
agent_event_missing_timestamp
metadata_unknown_field
note_item_missing_id

11.2 Forbidden native-note content

Noumena-native Phase 1 notes reject active or dangerous content:

- `<script>` except the inert metadata JSON script
- `<iframe>`
- `<object>`
- `<embed>`
- `<form>`
- `<input>`
- event handler attributes such as `onclick`
- `srcdoc`
- `javascript:`, `vbscript:`, and `file:` URL schemes
- `meta http-equiv="refresh"`

11.3 Research-note@1 validation rules

All five roles are required. There are no optional roles in Phase 1.

Allowed policies per role:

metadata: `merge_text`, `locked`
summary: `replace_requires_clean_base`, `manual_review`, `locked`
notes: `append_only`, `locked`
evidence: `append_only`, `locked`
agent_events: `append_only`, `locked`

Unknown roles are warnings unless duplicated. Duplicate roles are errors regardless of whether the role is known.

<title> mismatch with metadata.title is a warning. The next canonical serialization should reconcile the HTML <title> to metadata.title.

12 Revision and block hash rules

doc info is the authoritative way for Claude Code to discover the current revision. Claude Code should not parse HTML metadata to find a revision.

Native document revision:

```
sha256(canonicalSerialize(parseNoumenaHtml(fileBytes)))
```

Raw/imported HTML revision:

```
sha256(fileBytes)
```

Block hash:

```
sha256(canonicalSerializeBlock(block))
```

canonicalSerializeBlock(block) includes the canonical persistent block identity, attributes, and child content. It excludes nothing in Phase 1 unless explicitly declared non-canonical.

noumena:revision in HTML is forbidden.

13 doc info

Canonical command:

```
noumena doc info <vault> <doc-path> --json
```

Successful output for a native patchable file:

```
{
  "schema": "noumena.cli.result.v1",
  "command": "doc.info",
  "status": "ok",
  "document": "research/local-llms.html",
  "path": "research/local-llms.html",
  "documentId": "doc_01JZ8N4K6WQ7R9V2M3A5T1B0CX",
  "title": "Local LLM Memory Systems",
  "fileClass": "noumena_native",
  "patchable": true,
  "schemaVersion": "noumena.html.v1",
```

```

"documentType": "research-note",
"revision": "sha256:abc123",
"logicalVersion": 17,
"index": {
  "fresh": true,
  "indexedRevision": "sha256:abc123"
},
"roles": [
  {
    "role": "summary",
    "blockId": "blk_01JZSUMMARY0000000000000000",
    "policy": "replace_requires_clean_base",
    "blockHash": "sha256:sum111",
    "present": true,
    "known": true,
    "required": true,
    "requiresBlockHashForOps": ["propose_summary"]
  },
  {
    "role": "evidence",
    "blockId": "blk_01JZEVIDENCE00000000000000",
    "policy": "append_only",
    "blockHash": "sha256:ev111",
    "present": true,
    "known": true,
    "required": true,
    "requiresBlockHashForOps": []
  }
],
"allowedPhase1Ops": [
  "append_evidence",
  "append_note",
  "set_status",
  "set_metadata",
  "append_agent_event",
  "propose_summary"
]
}

```

allowedPhase1Ops is dynamic. Locked target blocks remove their associated operations.

For raw/imported or invalid files, doc info exits 0, returns a live revision, and sets patchable: false with a reason or validation errors.

Missing files, invalid vaults, path escapes, and uninitialized vaults exit 4.

14 doc create

Canonical command:

```
noumena doc create ./MyVault research/local-llms.html \  
--type=research-note \  
--title="Local LLM Memory Systems" \  
--json
```

Success result:

```
{  
  "schema": "noumena.cli.result.v1",  
  "command": "doc.create",  
  "status": "created",  
  "vault": "./MyVault",  
  "document": "research/local-llms.html",  
  "documentId": "doc_01JZ9A7V6QK8M3X2T5N4R1B0CY",  
  "title": "Local LLM Memory Systems",  
  "fileClass": "noumena_native",  
  "patchable": true,  
  "schemaVersion": "noumena.html.v1",  
  "documentType": "research-note",  
  "templateUsed": "research-note@1",  
  "revision": "sha256:abc123",  
  "logicalVersion": 1,  
  "indexUpdated": true,  
  "next": {  
    "infoCommand": "noumena doc info ./MyVault research/local-llms.html --json"  
  }  
}
```

Rules:

- vault init must have already succeeded.
- Parent directories inside the vault may be created.
- File must not already exist.
- --type must be research-note.
- No --overwrite flag in Phase 1.
- Created document must pass validateDocument() with no errors.

If the file exists:

```
{
  "schema": "noumena.cli.error.v1",
  "command": "doc.create",
  "status": "error",
  "error": {
    "code": "document_already_exists",
    "message": "A file already exists at the requested document path.",
    "path": "research/local-llms.html"
  },
  "documentChanged": false
}
```

Exit code: 4.

15 Patch protocol

Patch schema name: noumena.patch.v1.

Minimal patch:

```
{
  "schema": "noumena.patch.v1",
  "clientPatchId": "claude-code-local-llms-evidence-2026-05-29T23-00-00Z",
  "base": {
    "revision": "sha256:abc123",
    "logicalVersion": 17,
    "blockHashes": {
      "blk_01JZSUMMARY0000000000000000": "sha256:sum111"
    }
  },
  "author": {
    "type": "agent",
    "name": "Claude Code"
  },
  "intent": "Add evidence and propose an updated summary.",
  "ops": []
}
```

Required:

- schema
- base.revision
- author.type
- author.name

- intent
- ops

Recommended:

- clientPatchId
- base.logicalVersion
- base.blockHashes for any clean-base/manual-review target

clientPatchId provides idempotency. If the same clientPatchId was already applied, return status: "noop" with reason already_applied.

15.1 Phase 1 operations

Operation	Default target	Auto-apply?	Notes
append_evidence	role evidence	Yes	Append-only block
append_note	role notes	Yes	Plain text only
set_status	role metadata	Yes	status enum only
set_metadata	role metadata	Yes	only tags, aliases
append_agent_event	role agent_events	Yes	explicit op only
propose_summary	role summary	No	always queues review

propose_summary always queues review in Phase 1 even if the target block hash matches.

16 patch submit

Canonical command:

```
noumena patch submit <vault> <doc-path> <patch.json> --json
```

patch submit can return four valid domain statuses:

```
applied
queued_review
rejected
noop
```

All four exit 0.

16.1 Applied output

```
{
  "schema": "noumena.cli.result.v1",
```

```

"command": "patch.submit",
"status": "applied",
"patchId": "patch_01JZ8K7M4Y9F3A2Q6XW0N1B2C3",
"reviewId": null,
"document": "research/local-llms.html",
"documentChanged": true,
"previousRevision": "sha256:abc123",
"revision": "sha256:def456",
"snapshotPath":
".noumena/history/snapshots/research/local-llms.html/patch_01JZ8K7M4Y9F3A2Q6XW0N
1B2C3.before.html",
"diffPath": ".noumena/history/patches/patch_01JZ8K7M4Y9F3A2Q6XW0N1B2C3.diff",
"indexUpdated": true,
"next": {
  "rollbackCommand": "noumena patch rollback ./MyVault
patch_01JZ8K7M4Y9F3A2Q6XW0N1B2C3 --json"
}
}

```

16.2 Queued review output

```

{
  "schema": "noumena.cli.result.v1",
  "command": "patch.submit",
  "status": "queued_review",
  "patchId": "patch_01JZ8K9BWQAXD5E4G7H8J9K0LM",
  "reviewId": "rev_01JZ8K9D3CR7M1TY4P6V8Q2N5A",
  "document": "research/local-llms.html",
  "documentChanged": false,
  "baseRevision": "sha256:abc123",
  "currentRevision": "sha256:abc123",
  "diffPath": ".noumena/reviews/rev_01JZ8K9D3CR7M1TY4P6V8Q2N5A.diff",
  "next": {
    "approveCommand": "noumena patch approve ./MyVault
rev_01JZ8K9D3CR7M1TY4P6V8Q2N5A --json",
    "rejectCommand": "noumena patch reject ./MyVault
rev_01JZ8K9D3CR7M1TY4P6V8Q2N5A --json",
    "showCommand": "noumena patch show ./MyVault
rev_01JZ8K9D3CR7M1TY4P6V8Q2N5A --format=block-diff"
  }
}

```

16.3 Rejected output

```
{
  "schema": "noumena.cli.result.v1",
  "command": "patch.submit",
  "status": "rejected",
  "document": "research/local-llms.html",
  "documentChanged": false,
  "reason": {
    "code": "target_block_changed_since_base_revision",
    "message": "Target block changed since the patch base revision.",
    "blockId": "blk_01JZSUMMARY0000000000000000",
    "role": "summary",
    "policy": "replace_requires_clean_base",
    "expectedBlockHash": "sha256:sum111",
    "currentBlockHash": "sha256:sum222"
  }
}
```

16.4 No-op output

```
{
  "schema": "noumena.cli.result.v1",
  "command": "patch.submit",
  "status": "noop",
  "document": "research/local-llms.html",
  "documentChanged": false,
  "reason": {
    "code": "already_applied",
    "priorPatchId": "patch_01JZ8K7M4Y9F3A2Q6XW0N1B2C3"
  },
  "revision": "sha256:def456"
}
```

17 Conflict detection

Phase 1 uses both full document revisions and per-block hashes.

base.revision:

Required for every patch.

Identifies the full document Claude Code read.

base.blockHashes:

Required for operations targeting `replace_requires_clean_base` or `manual_review` blocks.

Keyed by block ID, not role.
Optional and ignored for append_only blocks.

17.1 Append-only policy

For append_only targets:

- full document revision mismatch does not reject
- target block must still exist
- target block must still be appendable
- locked targets reject
- emit warning base_revision_changed_append_merged when the full revision is stale

17.2 Clean-base and manual-review policies

For replace_requires_clean_base or manual_review targets:

- base.blockHashes[targetBlockId] is required
- current target block hash must match
- mismatch rejects with target_block_changed_since_base_revision during submit or stale_review during approval

17.3 Merge-text policy

For metadata in Phase 1:

- no block hash required
- only field-level allowlist operations are supported
- set_status edits status
- set_metadata edits only tags and aliases
- locked metadata rejects

17.4 Atomic compound patch rule

A patch is atomic:

- if every op can auto-apply, the whole patch applies
- if any op requires review, the whole patch queues for review without changing the file
- if any op rejects, the whole patch rejects without changing the file
- there is no partially_applied_review_queued status

Therefore a patch containing append_evidence plus propose_summary queues the entire patch. Approval applies both together after revalidation.

18 Review approval semantics

Queued review records must store:

- patch JSON
- patch ID
- review ID
- document path
- base revision
- base block hashes
- resolved target block IDs for every op
- diff path
- queued timestamp
- review status

On approval:

1. Acquire vault write lock.
2. Read current document from disk.
3. Validate current document.
4. Resolve stored target block IDs.
5. For append-only ops, ensure target still exists and is appendable.
6. For clean-base/manual-review ops, compare current block hash to stored base hash.
7. If any target block vanished or changed, reject approval with `stale_review`.
8. If all checks pass, snapshot, apply whole patch, serialize, write atomically, update index/history.

Evidence append should not stale a queued summary review if the summary block hash remains unchanged.

19 Write lock and atomic writes

Phase 1 CLI is standalone, so all Noumena writers must coordinate through a vault-level advisory write lock.

Lock path:

```
<vault>/noumena/locks/vault-write.lock/  
owner.json
```

Acquire lock by atomically creating the lock directory. If it exists and is fresh, return status: "rejected" with `reason.code: "vault_write_lock_busy"` and exit 0.

owner.json shape:

```
{
  "schema": "noumena.lock.v1",
  "owner": {
    "kind": "cli",
    "command": "patch.submit"
  },
  "pid": 12345,
  "startedAt": "2026-05-29T23:00:00Z",
  "expiresAt": "2026-05-29T23:01:00Z"
}
```

Stale locks are detected by expiry plus PID check where possible. If clearly stale, move the lock directory to:

`.noumena/locks/stale/vault-write.lock.<timestamp>/`

Do not silently delete stale lock evidence.

19.1 Mutation write sequence

All mutations use this sequence:

1. Acquire vault write lock.
2. Open SQLite transaction with BEGIN IMMEDIATE.
3. Read current file bytes from disk.
4. Parse and validate current document.
5. Compute current revision and block hashes.
6. Evaluate patch or command.
7. For rejected outcomes, record attempt if useful, commit, release lock, return.
8. For queued review, write review record and diff, commit, release lock, return.
9. For applied mutation, write pre-mutation snapshot.
10. Apply mutation in memory.
11. Canonical serialize.
12. Write temp file next to target.
13. fsync temp file where available.
14. Rename temp file over target.
15. Update SQLite index/history records.
16. Commit SQLite transaction.
17. Release lock.
18. Return JSON result.

If index update fails after file write, return status: "applied" or status: "created" with indexUpdated: false and a warning. Do not claim the patch failed if the HTML file was successfully written.

20 SQLite derived state

SQLite is derived state plus operational history. It is not the canonical document source.

Recommended minimal schema sketch:

```
CREATE TABLE documents (  
  path TEXT PRIMARY KEY,  
  document_id TEXT,  
  title TEXT,  
  file_class TEXT NOT NULL,  
  schema_version TEXT,  
  document_type TEXT,  
  revision TEXT NOT NULL,  
  logical_version INTEGER NOT NULL,  
  indexed_at TEXT NOT NULL,  
  modified_at TEXT  
);
```

```
CREATE TABLE blocks (  
  path TEXT NOT NULL,  
  block_id TEXT NOT NULL,  
  role TEXT NOT NULL,  
  policy TEXT NOT NULL,  
  block_hash TEXT NOT NULL,  
  required INTEGER NOT NULL,  
  known INTEGER NOT NULL,  
  PRIMARY KEY (path, block_id)  
);
```

```
CREATE TABLE patches (  
  patch_id TEXT PRIMARY KEY,  
  client_patch_id TEXT UNIQUE,  
  path TEXT NOT NULL,  
  status TEXT NOT NULL,  
  author_name TEXT,  
  author_type TEXT,  
  intent TEXT,  
  base_revision TEXT,  
  previous_revision TEXT,  
  revision TEXT,  
  patch_json_path TEXT,  
  diff_path TEXT,  
  snapshot_path TEXT,
```

```
created_at TEXT NOT NULL,  
applied_at TEXT  
);
```

```
CREATE TABLE reviews (  
  review_id TEXT PRIMARY KEY,  
  patch_id TEXT NOT NULL,  
  path TEXT NOT NULL,  
  status TEXT NOT NULL,  
  base_revision TEXT NOT NULL,  
  review_json_path TEXT NOT NULL,  
  diff_path TEXT NOT NULL,  
  queued_at TEXT NOT NULL,  
  resolved_at TEXT  
);
```

```
CREATE TABLE history_events (  
  event_id TEXT PRIMARY KEY,  
  path TEXT NOT NULL,  
  kind TEXT NOT NULL,  
  patch_id TEXT,  
  review_id TEXT,  
  revision_before TEXT,  
  revision_after TEXT,  
  created_at TEXT NOT NULL,  
  data_json TEXT  
);
```

Implementer may adjust this schema as long as the observable CLI behavior and vault doctor, patch status, and index freshness requirements are satisfied.

21 Canonical serialization

Canonical serialization must be deterministic enough for byte-for-byte golden fixture tests.

Global rules:

- lowercase tag names
- lowercase attribute names
- two-space HTML indentation
- LF line endings
- one trailing newline
- no trailing spaces

- block-level elements on their own lines
- inline elements stay inline where specified
- class tokens deduped and sorted lexicographically

Attribute order:

id
 sorted data-noumena-* attributes
 data-conflict-policy
 href
 src
 class
 other allowed attributes sorted alphabetically

Shell exceptions:

```
<html lang="en">
<meta charset="utf-8">
<meta name="noumena:schema" content="noumena.html.v1">
```

Metadata JSON:

- sorted keys recursively
- 2-space JSON indentation
- arrays one item per line
- no comments
- no trailing spaces
- JSON starts at column 1 inside the script block

canonicalSerialize(parseNoumenaHtml(bytes)) must be idempotent after the second pass.
 Golden fixtures fail CI if output changes.

22 Phase 1 mutation HTML shapes

22.1 append_evidence

Patch input:

```
{
  "op": "append_evidence",
  "target": {
    "role": "evidence"
  },
  "item": {
    "claim": "Local LLM tools often distinguish persistent memory from active context.",
    "source": {
```

```

    "title": "Example Source",
    "url": "https://example.com/source",
    "accessedAt": "2026-05-29T23:00:00Z"
  },
  "quote": "Short relevant excerpt.",
  "notes": "Supports the note's memory/context distinction."
}
}

```

Canonical output appended to evidence :

```

<li
  id="ev_01JZ9QK7J8V6B4N2M1P0X3Y5ZA"
  data-noumena-added-at="2026-05-29T23:00:00Z"
  data-noumena-added-by="Claude Code"
  data-noumena-evidence-id="ev_01JZ9QK7J8V6B4N2M1P0X3Y5ZA">
  <p data-noumena-field="claim" class="nfm-claim"><strong>Claim:</strong> <span
data-noumena-field="claim-text">Local LLM tools often distinguish persistent memory
from active context.</span></p>
  <p data-noumena-field="source" class="nfm-source"><strong>Source:</strong> <a data-
noumena-field="source-title" href="https://example.com/source">Example Source</a>
<time data-noumena-field="accessed-at" class="nfm-accessed-at" datetime="2026-05-
29T23:00:00Z">(accessed 2026-05-29T23:00:00Z)</time></p>
  <blockquote data-noumena-field="quote" class="nfm-quote">Short relevant
excerpt.</blockquote>
  <p data-noumena-field="notes" class="nfm-notes">Supports the note's memory/context
distinction.</p>
</li>

```

If URL is absent, render source title as a . If quote or notes are absent, omit those elements. Do not emit empty optional elements.

22.2 append_note

Patch input:

```

{
  "op": "append_note",
  "target": {
    "role": "notes"
  },
  "text": "This is a short appended note."
}

```

Canonical output:

```
<p
id="note_01JZ9QP7Z4N1F8K2D6C3V5A9BM"
data-noumena-added-at="2026-05-29T23:00:00Z"
data-noumena-added-by="Claude Code"
data-noumena-note-id="note_01JZ9QP7Z4N1F8K2D6C3V5A9BM"
class="nfm-note">This is a short appended note.</p>
```

append_note accepts plain text only. Text is HTML-escaped. If the notes section contains a placeholder <p></p>, remove it on first append.

22.3 append_agent_event

Patch input:

```
{
  "op": "append_agent_event",
  "target": {
    "role": "agent_events"
  },
  "event": {
    "eventType": "research_update",
    "summary": "Added one evidence item about local LLM memory.",
    "details": "Optional additional detail."
  }
}
```

Canonical output:

```
<li
id="evt_01JZ9QR5T8N4C7M2K9P1V6X3YA"
data-noumena-author="Claude Code"
data-noumena-event-id="evt_01JZ9QR5T8N4C7M2K9P1V6X3YA"
data-noumena-event-type="research_update"
data-noumena-timestamp="2026-05-29T23:00:00Z">
  <span data-noumena-field="event-summary" class="nfm-event-summary">Added one
evidence item about local LLM memory.</span>
  <p data-noumena-field="event-details" class="nfm-event-details">Optional additional
detail.</p>
</li>
```

If details is absent, omit the details paragraph.

Noumena does not automatically append agent events for every patch. Patch/history records already live in .noumena/history. Visible agent events change only through explicit append_agent_event ops.

22.4 set_status

Allowed values:

draft
active
reviewed
archived

set_status updates metadata.status and stamps metadata.updatedAt to the write timestamp on successful forward mutation.

22.5 set_metadata

Phase 1 allowed fields:

tags
aliases

Rules:

- both are arrays of strings
- trim strings
- remove empty strings
- deduplicate
- sort lexicographically
- reject attempts to modify title, createdAt, updatedAt, status, schema, or document ID fields

Forbidden field output:

```
{  
  "status": "rejected",  
  "reason": {  
    "code": "metadata_field_not_allowed",  
    "field": "createdAt"  
  }  
}
```

22.6 propose_summary

propose_summary always queues review during submit.

Patch input:

```
{  
  "op": "propose_summary",  
  "target": {
```

```
"role": "summary"
},
"content": {
  "html": "<p>Local LLM memory systems should separate persistent knowledge from
active conversational context.</p>"
}
}
```

Allowed summary fragment tags:

p
strong
em
code
a
br

Rules:

- one or more paragraphs required
- plain text may be converted to one paragraph
- no headings
- no script, iframe, object, form, input, event handlers, or srcdoc
- links may use HTTP, HTTPS, mailto, relative paths, or fragments
- forbidden URL schemes reject the patch

On approval, replace only the summary section content after the direct <h2>. Preserve the section element, attributes, block ID, role, policy, and heading.

23 Timestamp rules

Use one write timestamp per successful forward mutation.

metadata.updatedAt changes on:

- doc create
- patch submit with status: applied
- patch approve with status: applied

metadata.updatedAt does not change on:

- queued_review
- rejected
- noop
- rollback

- dry-run
- doc info
- doc validate
- patch show
- patch status
- vault doctor

Rollback restores the pre-patch snapshot exactly, including the old updatedAt.

24 patch show --format=block-diff

Phase 1 review UI is terminal output.

patch show should print a human-readable block diff. It does not need to be machine-stable JSON. Recommended behavior:

- ANSI color when stdout is a TTY
- plain text when piped
- show patch ID, review ID, document path, author, intent, base revision
- show affected roles and block IDs
- show before/after block text diff
- show raw proposed summary fragment for propose_summary

No AC depends on exact ANSI coloring.

25 Rollback

Phase 1 rollback is file-level and latest-patch-per-document only.

Command:

```
noumena patch rollback <vault> <patch-id> --json
```

Success:

```
{
  "schema": "noumena.cli.result.v1",
  "command": "patch.rollback",
  "status": "rolled_back",
  "patchId": "patch_01JZ8K7M4Y9F3A2Q6XW0N1B2C3",
  "rollbackId": "rollback_01JZ8KC4F6S2A9D8Q1M7N5B3V0",
  "document": "research/local-llms.html",
  "documentChanged": true,
  "previousRevision": "sha256:def456",
  "revision": "sha256:abc123",
```

```
"restoredFromSnapshot":  
".noumena/history/snapshots/research/local-llms.html/patch_01JZ8K7M4Y9F3A2Q6XW0N  
1B2C3.before.html",  
"indexUpdated": true  
}
```

Non-latest patch rollback returns exit 0 with:

```
{  
  "status": "rejected",  
  "reason": {  
    "code": "rollback_not_latest"  
  },  
  "documentChanged": false  
}
```

Rollback records a history event in .noumena/history, but does not stamp the restored HTML.

26 Vault doctor

`noumena vault doctor <vault> --json` detects recoverable local consistency issues:

- stale locks
- interrupted temp files
- index revision mismatches
- stale reviews
- missing snapshots referenced by history
- patch records whose files are missing

Example:

```
{  
  "schema": "noumena.cli.result.v1",  
  "command": "vault.doctor",  
  "status": "ok",  
  "vault": "./MyVault",  
  "issues": [  
    {  
      "code": "index_stale",  
      "message": "3 documents have changed since indexing.",  
      "fixCommand": "noumena index rebuild ./MyVault --json"  
    }  
  ]  
}
```

index rebuild is optional in Phase 1 unless implemented as a doctor fix command. The CLI should not require full search to satisfy Phase 1.

27 Core algorithms

27.1 doc info

```
async function docInfo(vaultPath: string, docPath: string): Promise<DocInfoResult> {
  const vault = await openInitializedVault(vaultPath);
  const safePath = validateVaultRelativePath(vault, docPath);
  const bytes = await readFile(safePath.absolute);

  const parsed = parseHtml(bytes);
  const classification = classifyFile(parsed);

  if (classification.fileClass !== "noumena_native") {
    return rawOrInvalidInfo(bytes, parsed, classification);
  }

  const doc = parseNoumenaHtml(bytes);
  const validation = validateDocument(doc);
  const revision = computeRevision(doc);
  const roles = validation.roles.map(role => ({
    ...role,
    blockHash: computeBlockHash(role.block)
  }));

  return {
    status: "ok",
    revision,
    patchable: validation.patchable,
    roles,
    allowedPhase1Ops: allowedPhase1Ops(validation)
  };
}
```

27.2 patch submit

```
async function submitPatch(args: PatchSubmitArgs): Promise<PatchSubmitResult> {
  const vault = await openInitializedVault(args.vaultPath);
  const docPath = validateVaultRelativePath(vault, args.docPath);
  const patch = await readAndValidatePatchJson(args.patchPath);

  const lock = await acquireVaultWriteLock(vault, "patch.submit");
  try {
```

```

const db = await openDb(vault);
await db.exec("BEGIN IMMEDIATE");

const beforeBytes = await readFile(docPath.absolute);
const currentDoc = parseNoumenaHtml(beforeBytes);
const validation = validateDocument(currentDoc);

if (!validation.patchable) {
  await recordRejectedPatchAttempt(db, patch, validation);
  await db.exec("COMMIT");
  return rejected("document_validation_failed", { errors: validation.errors });
}

const currentRevision = computeRevision(currentDoc);
const decision = evaluatePatch({ patch, currentDoc, currentRevision });

if (decision.status === "rejected") {
  await recordRejectedPatchAttempt(db, patch, decision);
  await db.exec("COMMIT");
  return rejectedResult(decision);
}

if (decision.status === "queued_review") {
  await writeReviewRecord(vault, db, patch, decision);
  await db.exec("COMMIT");
  return queuedReviewResult(decision);
}

const snapshotPath = await writeBeforeSnapshot(vault, docPath.relative, beforeBytes,
decision.patchId);
const nextDoc = applyPatchToDocument(currentDoc, decision);
const nextHtml = canonicalSerialize(nextDoc);
const nextRevision = computeRevision(nextDoc);

await atomicWriteFile(docPath.absolute, nextHtml);
const indexUpdated = await tryIndexDocument(db, docPath.relative, nextDoc,
nextRevision);
await recordAppliedPatch(db, decision, snapshotPath, currentRevision, nextRevision);
await db.exec("COMMIT");

return appliedResult(decision, { currentRevision, nextRevision, snapshotPath,
indexUpdated });
} catch (err) {

```

```

    await rollbackQuietly();
    throw err;
  } finally {
    await lock.release();
  }
}

```

27.3 evaluatePatch

```

function evaluatePatch(input: EvaluatePatchInput): PatchDecision {
  const opDecisions: OpDecision[] = [];

  for (const [index, op] of input.patch.ops.entries()) {
    const target = resolveTarget(input.currentDoc, op);

    if (!target) return reject("target_not_found", { opIndex: index });
    if (target.policy === "locked") return reject("target_block_locked", targetInfo(target));

    if (target.policy === "append_only") {
      opDecisions.push({ status: "auto_apply", opIndex: index, targetBlockId:
target.blockId });
      continue;
    }

    if (target.policy === "replace_requires_clean_base" || target.policy ===
"manual_review") {
      const expected = input.patch.base.blockHashes?.[target.blockId];
      if (!expected) return reject("missing_required_block_hash", targetInfo(target));

      const current = computeBlockHash(target.block);
      if (expected !== current) {
        return reject("target_block_changed_since_base_revision", {
          ...targetInfo(target),
          expectedBlockHash: expected,
          currentBlockHash: current
        });
      }

      opDecisions.push({ status: "requires_review", opIndex: index, targetBlockId:
target.blockId });
      continue;
    }

    if (target.policy === "merge_text") {

```

```

    const result = evaluateMetadataMerge(op, target);
    if (!result.ok) return reject(result.reason.code, result.reason);
    opDecisions.push({ status: "auto_apply", opIndex: index, targetBlockId:
target.blockId });
  }
}

if (opDecisions.some(d => d.status === "requires_review")) {
  return { status: "queued_review", opDecisions };
}

return { status: "applied", opDecisions };
}

```

28 Security and safety in Phase 1

Phase 1 has no Electron, no localhost server, no IPC, and no untrusted rendered webview. The security work is therefore focused on file safety, patch safety, and HTML safety.

Required Phase 1 safety controls:

- canonicalize and validate vault paths
- reject path escapes and symlink escapes
- reject writes inside .noumena/ through document-path arguments
- use advisory vault write lock for Noumena writers
- validate all patches against noumena-patch-v1.schema.json
- reject forbidden HTML in Noumena-native notes
- escape all agent-provided text for append operations
- validate restricted summary fragments before storing reviews
- reject dangerous URL schemes
- reject arbitrary metadata field writes
- store provenance in patch/review/history records
- never treat note content as instructions to the CLI

Phase 2 adds Electron security defaults. Phase 1 should not front-load Electron-specific CSP, preload, IPC sender validation, or sandbox work.

29 Testing and fixtures

Phase 1 is test-driven by fixtures. The fixture corpus is part of the product contract.

Required fixture directories:

fixtures/

native/

research-note-empty.html
research-note-with-evidence.html
research-note-with-agent-events.html

malformed/

missing-required-role.html
duplicate-role.html
duplicate-block-id.html
invalid-metadata-json.html

validation/

valid/

research-note-empty.valid.html
research-note-with-evidence.valid.html
research-note-with-warning-title-mismatch.valid.html
research-note-with-unknown-role.valid.html

invalid/

missing-doctype.invalid.html
missing-schema-meta.invalid.html
unknown-schema-version.invalid.html
missing-document-article.invalid.html
multiple-document-articles.invalid.html
invalid-document-id.invalid.html
forbidden-revision-meta.invalid.html
duplicate-block-id.invalid.html
duplicate-role.invalid.html
missing-metadata-role.invalid.html
invalid-metadata-json.invalid.html
missing-summary-role.invalid.html
missing-notes-role.invalid.html
missing-evidence-role.invalid.html
missing-agent-events-role.invalid.html
evidence-missing-ol.invalid.html
agent-events-missing-ol.invalid.html
unknown-conflict-policy.invalid.html
policy-not-allowed-for-role.invalid.html
forbidden-script.invalid.html
forbidden-iframe.invalid.html
javascript-url.invalid.html

patches/

append-evidence.valid.json
append-note.valid.json
set-status.valid.json
set-metadata.valid.json
append-agent-event.valid.json
propose-summary.queues-review.json
stale-base.rejects.json
invalid-schema.rejects.json

expected/
append-evidence.after.html
append-note.after.html
append-agent-event.after.html
set-status.after.html
set-metadata.after.html
approve-propose-summary.after.html
rollback-restores-original.after.html

Golden tests compare outputs byte-for-byte.

Required test commands:

pnpm test:phase1
pnpm demo:phase1

pnpm test:phase1 must run without Electron or a browser.

30 Phase 1 acceptance criteria

30.1 AC-001 - Patch submit CLI contract

noumena patch submit <vault> <doc> <patch.json> --json returns exactly one JSON object with status in applied, queued_review, rejected, or noop. Valid domain outcomes exit 0. Invalid input/internal failures use exit codes 2, 3, 4, or 5.

Verification:

- clean append apply
- queued review approve
- stale-base reject
- lock-busy reject
- idempotent noop
- compound patch queues atomically

30.2 AC-002 - Document info contract

noumena doc info returns live disk revision, logical version, document ID, file class, patchability, roles, block hashes, dynamic allowed ops, and index freshness. Claude Code can construct a valid patch without parsing HTML.

Verification:

- native file
- raw file
- invalid file
- missing file
- path escape
- block hash stability
- locked blocks excluded from allowed ops

30.3 AC-003 - Standalone CLI and write lock

All Phase 1 commands work without Electron. All writers use the vault-level advisory lock. Concurrent writer attempts return structured domain rejection.

Verification:

- two concurrent CLI invocations
- stale lock handling
- crash-and-recover through vault doctor
- index update failure returns warning, not command failure

30.4 AC-004 - Vault init and document create

vault init creates the required .noumena/ structure. doc create creates a valid research-note@1 without Electron, with all five required roles and no overwrite support.

Verification:

- create then validate
- create then info
- duplicate create fails
- path escape fails
- non-HTML path fails
- unsupported type fails

30.5 AC-005 - Phase 1 done condition

pnpm test:phase1 passes from a clean checkout with no Electron process. It covers the complete headless dogfood loop and all critical negative cases.

Required scenarios:

1. vault init succeeds
2. doc create produces patchable research-note@1
3. doc info returns revision, roles, block hashes
4. clean patch apply succeeds
5. queued review works
6. patch show prints block diff
7. approve review applies
8. reject review dismisses
9. stale review rejects
10. stale base rejects
11. evidence append does not stale summary review
12. idempotent noop works
13. lock busy rejects
14. path escape rejects
15. missing doc errors
16. malformed patch schema exits 3
17. rollback latest restores original revision
18. rollback non-latest rejects
19. invalid Noumena file is patchable false
20. manually placed valid fixture is patchable true
21. compound append plus propose queues atomically and approval applies both

30.6 AC-006 - Validation rules

validateDocument() implements two-layer validation for Noumena HTML v1 and research-note@1. Errors block patching; warnings do not.

Verification:

- all invalid validation fixtures produce expected error codes
- all valid fixtures pass
- patchable: true only when validation is clean

30.7 AC-007 - Conflict model

Conflict detection uses full document revision and per-block hashes. Append-only ops merge through stale full document revisions. Clean-base/manual-review ops require matching target block hashes.

Verification:

- evidence append does not stale summary review
- summary edit stales summary review
- missing block hash rejects
- append ignores stale full revision

- compound patch queues atomically
- locked target rejects

30.8 AC-008 - Canonical mutation serialization

Every write operation has exact canonical HTML output. Golden fixtures verify `append_evidence`, `append_note`, `append_agent_event`, `set_status`, `set_metadata`, `approved`, `propose_summary`, and `rollback` restoration.

Verification:

- byte-for-byte golden fixture comparison
- round-trip idempotence
- text escaping tests
- forbidden summary fragment rejects
- `rollback` restores pre-patch `updatedAt`

31 Implementation sequence

The build should proceed in this order. Do not start Electron until the Phase 1 test suite passes.

31.1 Step 1 - Repository skeleton

Deliverables:

- workspace setup
- `packages/noumena-core`
- `packages/noumena-cli`
- test runner
- basic CLI entrypoint
- deterministic test clock and ID generator

Done when:

```
pnpm install
```

```
pnpm build
```

```
pnpm test:phase1
```

runs an empty/placeholder suite successfully.

31.2 Step 2 - Schemas and fixtures scaffolding

Deliverables:

- `schemas/noumena-html-v1.md`
- `schemas/noumena-patch-v1.schema.json`
- `schemas/noumena-cli-result-v1.schema.json`

- initial fixtures directories
- fixture loading helpers

Done when schema validation can fail malformed patch JSON with exit 3.

31.3 Step 3 - Path safety and vault init

Deliverables:

- openInitializedVault
- path canonicalization
- symlink escape checks
- vault init
- .noumena/ directory creation
- initial SQLite database
- .noumena/.gitignore

Done when path escape and uninitialized vault tests pass.

31.4 Step 4 - Parser and serializer

Deliverables:

- parseNoumenaHtml
- canonicalSerialize
- canonicalSerializeBlock
- computeRevision
- deterministic attribute sorting
- metadata JSON sorting

Done when round-trip idempotence tests pass for initial valid fixtures.

31.5 Step 5 - Validation

Deliverables:

- structural validation
- research-note@1 validation
- forbidden content checks
- role/policy matrix
- dynamic allowed ops
- doc validate

Done when all validation fixtures pass with expected error/warning codes.

31.6 Step 6 - Document create and info

Deliverables:

- doc create
- research-note@1 creation path
- deterministic IDs in tests
- doc info
- block hash calculation
- patchable classification

Done when create/info AC-002 and AC-004 tests pass.

31.7 Step 7 - Locking, transactions, and history primitives

Deliverables:

- vault write lock
- stale lock archive
- SQLite BEGIN IMMEDIATE
- atomic file write helper
- snapshot writer
- patch/review/history record writers
- vault doctor minimal checks

Done when lock-busy and basic crash/recovery tests pass.

31.8 Step 8 - Patch evaluation

Deliverables:

- patch schema validation
- target resolution by role and block ID
- conflict policy evaluator
- full revision plus block hash logic
- atomic compound patch decisions
- idempotency by clientPatchId

Done when AC-007 conflict tests pass without actual file mutation.

31.9 Step 9 - Patch mutations and canonical outputs

Deliverables:

- append_evidence
- append_note
- append_agent_event

- set_status
- set_metadata
- propose_summary review fragment handling
- metadata updatedAt stamping
- HTML escaping and summary fragment sanitizer

Done when AC-008 golden fixtures pass.

31.10 Step 10 - Patch CLI commands

Deliverables:

- patch dry-run
- patch submit
- patch approve
- patch reject
- patch rollback
- patch status
- patch show --format=block-diff

Done when AC-001 tests pass.

31.11 Step 11 - Full integration suite and demo

Deliverables:

- pnpm test:phase1
- pnpm demo:phase1
- 21 AC-005 scenarios
- review diff fixture inspection
- final README handoff

Done when the suite passes from a clean checkout and the demo prints a final success JSON.

32 Error handling matrix

Phase 1 distinguishes invalid command/input failures from valid domain rejections. Valid domain rejections exit 0 and return status: "rejected" or another documented domain status. Invalid input or internal failures use non-zero exit codes.

32.1 Domain rejections: exit 0

- Vault write lock busy: status: "rejected", reason.code: "vault_write_lock_busy".
- Document validation fails: status: "rejected", reason.code: "document_validation_failed".

- Missing required block hash: status: "rejected", reason.code: "missing_required_block_hash".
- Target block locked: status: "rejected", reason.code: "target_block_locked".
- Stale base block hash: status: "rejected", reason.code: "target_block_changed_since_base_revision".
- Stale review: patch approve returns status: "rejected", reason.code: "stale_review".
- Rollback non-latest: status: "rejected", reason.code: "rollback_not_latest".
- Index update failed after a successful write: return status: "applied" or status: "created", indexUpdated: false, and a warning.

32.2 Input or system failures: non-zero exit

- Document not found: exit 4 with error.code: "document_not_found".
- Path escape: exit 4 with error.code: "path_escapes_vault".
- Malformed patch JSON: exit 3 with error.code: "patch_schema_invalid" and details[].
- Vault not initialized: exit 4 with error.code: "vault_not_initialized".
- File already exists on create: exit 4 with error.code: "document_already_exists".
- Internal failure: exit 5 with error.code: "internal_error".

33 Open questions accepted as non-blocking

These questions are recorded for the implementer, but none should block Phase 1 AC completion.

- OQ-001: Node.js version and package manager. Recommended answer: Node 20 LTS plus pnpm; implementer decides.
- OQ-002: ANSI color in patch show. Recommended answer: color for TTY, plain text when piped.
- OQ-003: .gitignore inside .noumena/. Recommended answer: yes; ignore locks/ and *.tmp.
- OQ-004: Full SQLite schema. Recommended answer: derive from the observable AC requirements and support vault doctor, patch status, and index freshness.
- OQ-005: patch show output format. Recommended answer: show a semantic before/after text diff plus raw proposed HTML for inspection.

34 Future compatibility requirements

Even though Phase 1 is CLI-only, it must not paint Phase 2 and Phase 3 into a corner.

Requirements:

- files created in Phase 1 are readable by Phase 2 Electron without migration
- noumena-core exposes APIs the Electron app can call instead of reimplementing

- noumena.patch.v1 can be extended backward-compatibly
- role/block schema remains compatible with future Tiptap node schema
- .noumena/ history and review records can be rendered by future UI
- no Phase 1 code assumes terminal-only forever; it just does not build GUI yet

35 Final recommendation

Build Noumena Phase 1 as a deterministic CLI-first core, not a desktop app. The Phase 1 product is the proof loop: a structured HTML research note, safe semantic patching by Claude Code, deterministic conflict handling, queued review, snapshots, rollback, and headless tests.

Do not start Electron until the Phase 1 CLI suite passes. Do not start Tiptap until the Electron viewer/review shell can wrap the CLI-created state. Do not add formats, importers, dashboards, plugins, or MCP until the basic human-agent document loop is real and dogfooded.

The implementation should be boring, strict, and test-heavy. That is the product advantage: agents can change local knowledge artifacts safely because every mutation is schema-validated, revisioned, conflict-aware, serialized deterministically, and reversible.